

TRAINING GAME — CONTOH KONTEN UJI

AI Fundamentals for the Workplace

Walkthrough lengkap satu sesi training, ditulis persis mengikuti schema & blueprint aplikasi

8 fase

3 role: host / central / player

Semua tipe PhaseContent dipakai

JSON valid + narasi per-role

Dokumen ini memakai setiap tipe fase dari **SHARED_SCHEMA** minimal sekali: microlearning, quiz (dua mode), video, content, codepiece, codeinput, presentation, idle, minigame. Setiap fase ditulis sebagai objek Phase yang benar-benar valid terhadap schema, lalu dijelaskan dari sudut pandang ketiga perangkat. Tujuannya: menguji apakah schema sungguh bisa merepresentasikan training nyata sebelum satu baris kode ditulis.

Cara membaca dokumen ini

Tiap fase punya tiga bagian: (1) objek Phase dalam JSON — inilah yang akan diauthoring di CMS dan dimuat runtime; (2) tiga kartu yang menjelaskan apa yang tampil & bisa dilakukan di layar Host, Central, dan Player; (3) catatan sinkronisasi yang menunjukkan node RTDB yang berubah saat fase berjalan.

Ingat aturan inti dari blueprint: runtime tidak membaca konten dari Firestore saat live — semua JSON di bawah ini sudah "dibekukan" ke dalam satu `publishedGames/{gameVersionId}` saat publish. Yang berubah saat sesi berjalan hanyalah node `/sessions/{id}/live/*` di RTDB. `syncMode` per fase menentukan apakah pemain digerakkan bersama (`lockstep`) atau maju sendiri-sendiri (`self_paced`).

Peta sesi — urutan default (phaseOrder)

#	Fase	type	syncMode	Inti aktivitas
0	Lobby / Pemanasan	idle	self_paced	Peserta join, animasi Lottie, host menunggu penuh
1	Selamat Datang & Tujuan	presentation	lockstep	Central jadi layar utama, host klik maju
2	Apa itu LLM?	microlearning	self_paced	Modul langkah-demi-langkah di HP masing-masing
3	Nonton: AI di alur kerja	video	lockstep	Video diputar bareng di central
4	Kuis Cepat (gaya Kahoot)	quiz · central_prompt	lockstep	Soal di central, tombol jawab di HP, timer
5	Refleksi Pribadi	quiz · on_device	self_paced	Soal terbuka dijawab tenang di HP
6	Mini-game: Urutkan Prompt	minigame · sort_order	lockstep	Seret langkah prompting ke urutan benar
7	Kunci Tim: Rakit Kode	codepiece + codeinput	lockstep	Tiap orang dapat potongan, dirakit di central

Karena host bisa membuka fase mana pun (modular), urutan ini hanya default. Fase 5 misalnya bisa dilewati kalau waktu mepet.

0

Lobby / Pemanasan

Peserta bergabung sebelum training dimulai

IDLE

SELF_PACED

```
// Phase - idle. Aman dirender di semua role, default saat phasePointer belum menunjuk konten.
{
  "id": "phase-0-lobby",
  "type": "idle",
  "title": "Lobby / Pemanasan",
  "syncMode": "self_paced",
  "roles": {
    "player": { "enabled": true },
    "central": { "enabled": true },
    "host": { "monitor": ["presence"] }
  },
  "content": {
    "type": "idle",
    "lottieMediaId": "media-lottie-brain-wave",
    "caption": "Menunggu peserta lain bergabung..."
  }
}
```

HOST

Membuat sesi, dapat **join code** 6-karakter, set maks peserta.

- Lihat daftar peserta masuk secara real-time (nama + status connected)
- Tombol "Mulai Training" aktif setelah cukup peserta

CENTRAL

Layar besar di ruangan menampilkan:

- Animasi Lottie "otak & gelombang"
- Join code besar + QR agar peserta gampang masuk
- Hitungan peserta yang sudah join

PLAYER

Di HP peserta:

- Masukkan nama, gabung via kode/QR
- Lihat Lottie yang sama + "Kamu sudah masuk, tunggu ya"

Kenapa self_paced di sini? Tidak ada langkah yang perlu disinkronkan — tiap orang hanya join. Host tidak menggerakkan siapa pun; ia hanya menunggu presence terisi.

RTDB berubah:

```
/sessions/{id}/presence/players/{playerId} = { name, connected:true, joinedAt }
/sessions/{id}/meta/status = "lobby" → (saat host klik Mulai) "live"
/sessions/{id}/live/phasePointer = { activePhaseId: "phase-1-welcome" }
```

1

Selamat Datang & Tujuan

Central jadi layar presentasi, host mengontrol slide

PRESENTATION

LOCKSTEP

```
// Presentation dikontrol host (controlledBy: "host"). Central = layar tayang, player pasif.
{
  "id": "phase-1-welcome",
  "type": "presentation",
  "title": "Selamat Datang & Tujuan",
  "syncMode": "lockstep",
  "roles": {
    "central": { "enabled": true },
    "player": { "enabled": true },
    "host": { "monitor": ["presence"] }
  },
  "content": {
    "type": "presentation",
    "controlledBy": "host",
    "slides": [
      { "id": "s1", "blocks": [
        { "kind": "text", "markdown": "# Selamat datang di AI Fundamentals" },
        { "kind": "image", "mediaId": "media-hero-team-ai" } ] },
      { "id": "s2", "blocks": [
        { "kind": "text", "markdown": "## Yang akan kita capai hari ini\n- Pahami cara kerja AI generatif\n- Tahu batas & risikonya\n- Bisa menyusun prompt yang baik" } ] }
    ]
  }
}
```

HOST

- Kontrol slide: tombol **Maju / Mundur** (karena controlledBy: "host")
- Melihat indikator slide berapa dari berapa
- Pantau bahwa semua central & player mengikuti

CENTRAL

- Menampilkan slide berukuran penuh, mengikuti centralStep
- Transisi otomatis saat host klik maju

PLAYER

- Layar "Lihat ke depan 👁️" + judul slide yang sedang tampil
- Tidak ada kontrol — peserta menyimak layar utama

Peran host di sini spesial. Biasanya slide dikontrol central sendiri (controlledBy: "central"), tapi di sesi berpemandu, host yang pegang kendali dari perangnya. Schema mendukung keduanya lewat satu field.

RTDB berubah:

```
/sessions/{id}/live/phasePointer = { activePhaseId: "phase-1-welcome" }
/sessions/{id}/live/centralStep = { step: 0 } → { step: 1 } // ditulis host
```

```
// Mode sequential = gaya onboarding, tombol Next digerbangi sampai pertanyaan dijawab.
{
  "id": "phase-2-what-is-llm",
  "type": "microlearning",
  "title": "Apa itu LLM?",
  "syncMode": "self_paced",
  "roles": {
    "player": { "enabled": true },
    "host": { "monitor": ["progress"] }
  },
  "content": {
    "type": "microlearning",
    "mode": "sequential",
    "steps": [
      { "id": "m1", "blocks": [
        { "kind": "text", "markdown": "***LLM** (Large Language Model) memprediksi kata berikutnya berdasar pola dari data teks yang sangat besar." },
        { "kind": "image", "mediaId": "media-next-token" } ] },
      { "id": "m2", "blocks": [
        { "kind": "text", "markdown": "Ia **tidak mengambil dari database fakta** – ia menghasilkan teks yang paling mungkin. Karena itu bisa terdengar meyakinkan tapi salah (*halusinasi*)." } ] },
      { "id": "m3", "gate": { "requireAnswered": true }, "blocks": [
        { "kind": "question", "question": {
          "qType": "single_choice",
          "prompt": [{ "kind": "text", "markdown": "Kenapa AI bisa memberi jawaban salah yang terdengar yakin?" }],
          "options": [
            { "id": "a", "label": "Ia memprediksi teks yang mungkin, bukan mengambil fakta" },
            { "id": "b", "label": "Database-nya sedang error" },
            { "id": "c", "label": "Ia sengaja berbohong" } ],
          "correctId": "a" } } ] }
    ]
  }
}
```

HOST

- Dashboard progres: "12/18 peserta selesai langkah 3"
- Tidak memaksa langkah — hanya memantau. Bisa memutuskan lanjut ke fase 3 saat mayoritas selesai

CENTRAL

- Tidak aktif (role central tidak enabled)
- Bisa menampilkan idle/progres ruangan sebagai opsi

PLAYER

- Baca kartu 1 → 2 → 3, tekan **Lanjut** sendiri
- Di langkah 3, tombol Lanjut terkunci sampai soal dijawab (gate)
- Kecepatan tiap orang beda — ini inti self_paced

Kontras penting dengan fase 1. Di sini `playerSharedStep` tidak dipakai; tiap HP menulis `players/{id}/selfStep` miliknya. Host melihat sebaran progres, bukan satu angka.

RTDB berubah (per pemain, tercakup sempit):

`/sessions/{id}/live/players/{playerId}/selfStep = 0 → 1 → 2`

`/sessions/{id}/live/players/{playerId}/status = "active" → "done"`

`/sessions/{id}/live/players/{playerId}/answers/m3 = { value:"a", submittedAt }`

3

Nonton Bareng: AI di Alur Kerja

Video diputar di central, semua menonton bersama

VIDEO

LOCKSTEP

```
{
  "id": "phase-3-video",
  "type": "video",
  "title": "AI di Alur Kerja Sehari-hari",
  "syncMode": "lockstep",
  "roles": {
    "central": { "enabled": true },
    "player": { "enabled": true },
    "host": { "monitor": ["presence"] }
  },
  "content": {
    "type": "video",
    "mediaId": "media-video-ai-workflow",
    "target": ["central"],
    "allowPlayerControl": false
  }
}
```

target: ["central"] = nonton bareng di layar besar. Kalau diisi ["player"], video tampil di tiap HP untuk ditonton mandiri.

HOST

- Kontrol putar / jeda video (karena player tidak boleh kontrol)
- Lanjut ke kuis setelah video selesai

CENTRAL

- Video layar penuh dengan audio ruangan

PLAYER

- Layar "Menonton bersama — lihat ke layar utama"
- Tidak ada pemutar di HP (allowPlayerControl:false)

RTDB berubah:

```
/sessions/{id}/live/phasePointer = { activePhaseId: "phase-3-video" }
```

```
/sessions/{id}/live/centralStep = { step: 0 } // state play/pause bisa disimpan di sini bila perlu
```

Kuis Cepat (gaya Kahoot)

Soal di central + timer, tombol jawab di HP, jumlah jawaban real-time

QUIZ · CENTRAL_PROMPT

LOCKSTEP

TIMER 20S

```
// mode central_prompt: SOAL di central, HP cuma tombol jawaban. Timer authority = server.
{
  "id": "phase-4-quiz-live",
  "type": "quiz",
  "title": "Kuis Cepat",
  "syncMode": "lockstep",
  "timer": { "seconds": 20, "authority": "server",
    "autoAdvanceOnExpire": true, "visibleTo": ["central", "player"] },
  "scoring": { "mode": "correctness_and_speed", "maxPoints": 1000,
    "speedBonus": { "maxBonus": 500, "decaySeconds": 20 } },
  "roles": {
    "central": { "enabled": true, "showTimer": true, "showResults": true },
    "player": { "enabled": true, "showTimer": true },
    "host": { "monitor": ["answers", "scores"] }
  },
  "content": {
    "type": "quiz",
    "mode": "central_prompt",
    "revealAnswers": true,
    "perQuestionTimerSeconds": 20,
    "questions": [
      { "qType": "single_choice",
        "prompt": [{ "kind": "text", "markdown": "Data rahasia perusahaan sebaiknya..." }],
        "options": [
          { "id": "a", "label": "Boleh ditempel ke AI publik asal cepat" },
          { "id": "b", "label": "Jangan ditempel ke AI publik" },
          { "id": "c", "label": "Boleh kalau dihapus setelahnya" } ],
        "correctId": "b" }
    ]
  }
}
```

HOST

- Tombol "Tampilkan soal" → memulai timer server
- Lihat jumlah yang sudah menjawab + distribusi jawaban
- Tombol "Reveal jawaban" (atau otomatis saat timer habis)

CENTRAL

- Soal + 3 opsi berwarna, **timer besar** hitung mundur
- Bar "14 dari 18 sudah menjawab" (dari answeredCount)
- Saat reveal: opsi benar disorot + leaderboard

PLAYER

- Hanya 3 **tombol warna** tanpa teks soal (soal ada di central)
- Timer kecil, feedback "Jawaban terkirim"
- Setelah reveal: benar/salah + poin (cepat = bonus)

Dua jebakan yang ditangani schema di sini. (1) **Timer** pakai authority: "server" → satu endsAt ditulis sekali, semua perangkat menghitung dari situ dengan koreksi serverTimeOffset, jadi HP lambat & layar tidak beda angka. (2) **Kunci jawaban** (correctId: "b") tidak boleh ikut ke bundle HP karena kuis ini bernilai — pengecekan lewat callable Function, kalau tidak jawaban kebaca di devtools.

RTDB berubah:

```
/sessions/{id}/live/timers/phase-4 = { endsAt: 1730000020000 } // epoch ms, sekali tulis
/sessions/{id}/live/players/{playerId}/answers/q0 = { value:"b", submittedAt }
/sessions/{id}/live/aggregates/answeredCount/q0 = 14 // via TRANSAKSI, bukan tulis langsung
/sessions/{id}/live/aggregates/scores/{playerId} = 1420 // dihitung server saat reveal
```

```
// mode on_device: soal DAN kolom jawab ada di HP. Tidak bernilai (participation), jadi aman inline.
{
  "id": "phase-5-reflection",
  "type": "quiz",
  "title": "Refleksi Pribadi",
  "syncMode": "self_paced",
  "scoring": { "mode": "participation" },
  "roles": {
    "player": { "enabled": true },
    "host": { "monitor": ["answers", "progress"] }
  },
  "content": {
    "type": "quiz",
    "mode": "on_device",
    "revealAnswers": false,
    "questions": [
      { "qType": "open_text", "maxLen": 280,
        "prompt": [{ "kind": "text", "markdown": "Satu tugas rutinmu yang bisa dibantu AI minggu ini?" }],
        "rubricHint": "Cari jawaban konkret & spesifik, bukan 'semua tugas'." },
      { "qType": "scale", "min": 1, "max": 5,
        "labels": ["Belum yakin", "Sangat yakin"],
        "prompt": [{ "kind": "text", "markdown": "Seberapa yakin kamu memakai AI dengan aman?" }] }
    ]
  }
}
```

HOST

- Baca jawaban terbuka yang masuk (untuk dibahas)
- Lihat rata-rata skala kepercayaan diri ruangan

CENTRAL

- Tidak aktif; opsional menampilkan word-cloud jawaban bila mau

PLAYER

- Isi jawaban terbuka (maks 280 karakter) + geser skala 1–5
- Tenang, tanpa timer — tekan Kirim saat siap

Kontras dengan fase 4. Sama-sama "quiz", tapi mode & syncMode berbeda total menghasilkan pengalaman berbeda. Inilah kenapa quiz butuh dua field terpisah, bukan satu toggle. Karena `scoring.mode: "participation"`, jawaban open-text ditulis saat submit/debounce (bukan per ketikan) ke node pemain sendiri.

RTDB berubah:

```
/sessions/{id}/live/players/{playerId}/answers/q0 = { value:"...", submittedAt } // on submit
/sessions/{id}/live/players/{playerId}/answers/q1 = { value:4, submittedAt }
```


Mini-game: Urutkan Prompt

Template `sort_order` — seret 4 langkah menyusun prompt yang baik

MINIGAME · SORT_ORDER

LOCKSTEP

TIMER 45S

```
// minigame = pluggable. templateId harus terdaftar di runtime. config divalidasi Zod milik template.
{
  "id": "phase-6-minigame-sort",
  "type": "minigame",
  "title": "Urutkan Langkah Prompting",
  "syncMode": "lockstep",
  "timer": { "seconds": 45, "authority": "server", "visibleTo": ["central", "player"] },
  "scoring": { "mode": "correctness_and_speed", "maxPoints": 800 },
  "roles": {
    "player": { "enabled": true, "showTimer": true },
    "central": { "enabled": true, "showResults": true },
    "host": { "monitor": ["scores"] }
  },
  "content": {
    "type": "minigame",
    "templateId": "sort_order",
    "config": {
      "items": [
        { "id": "i1", "label": "Tentukan tujuan & konteks" },
        { "id": "i2", "label": "Beri contoh format yang diinginkan" },
        { "id": "i3", "label": "Minta AI menghasilkan draft" },
        { "id": "i4", "label": "Periksa & perbaiki hasil" } ],
      "correctOrder": ["i1", "i2", "i3", "i4"]
    }
  }
}
```

HOST

- Mulai game, pantau skor masuk
- Lanjut ke fase penutup saat semua selesai / timer habis

CENTRAL

- Timer besar + papan skor langsung siapa tercepat & benar

PLAYER

- 4 kartu acak, **seret** ke urutan benar
- Kirim → skor dihitung (benar + cepat)

Batas jujur soal mini-game. Config di atas bisa diauthoring bebas di CMS — tapi hanya karena runtime **sudah punya kode** template `sort_order`. Bikin jenis mini-game baru (mis. matching pairs) tetap butuh kode di kedua repo, bukan sekadar isian CMS. "Mini-game tak terbatas dari CMS" itu mitos; "banyak instance dari template yang ada" itu nyata.

RTDB berubah:

```
/sessions/{id}/live/players/{playerId}/answers/sort = { order:["i1","i2","i3","i4"], submittedAt }
/sessions/{id}/live/aggregates/scores/{playerId} = 760
```

7

Kunci Tim: Rakit Kode

Dua fase berpasangan — codepiece (di HP) + codeinput (di central)

CODEPIECE → CODEINPUT

LOCKSTEP

Ini menunjukkan dua fase yang saling terkait. Tiap peserta menerima **potongan kode** di HP; ruangan harus menggabungkannya dan memasukkan hasil rakitan di **central screen** untuk "membuka kunci" dan menyelesaikan sesi.

7a — CodePiece (potongan di HP tiap peserta)

```
{
  "id": "phase-7a-codepiece",
  "type": "codepiece",
  "title": "Potongan Kodemu",
  "syncMode": "lockstep",
  "roles": { "player": { "enabled": true }, "host": { "monitor": ["presence"] } },
  "content": {
    "type": "codepiece",
    "distribution": "round_robin",
    "hint": "Gabungkan potongan seluruh tim sesuai nomor urutnya.",
    "fragments": [
      { "id": "f1", "value": "SAFE" },
      { "id": "f2", "value": "9" },
      { "id": "f3", "value": "AI" },
      { "id": "f4", "value": "24" } ]
  }
}
```

7b — CodeInput (rakitan diverifikasi di central)

```
{
  "id": "phase-7b-codeinput",
  "type": "codeinput",
  "title": "Buka Kunci Tim",
  "syncMode": "lockstep",
  "roles": { "central": { "enabled": true }, "host": { "monitor": ["presence"] } },
  "content": {
    "type": "codeinput",
    "expected": "SAFE9AI24",
    "caseSensitive": false,
    "maxAttempts": 5,
    "sourcePhaseId": "phase-7a-codepiece",
    "onSuccess": { "advance": true }
  }
}
```

HOST

- Pantau apakah kode sudah terpecahkan
- Bila mentok, bisa beri petunjuk lisan

CENTRAL

- Kolom input besar + sisa percobaan (maks 5)
- Salah → animasi getar; benar → "Terbuka!" + layar rangkuman sesi

PLAYER

- Kartu besar berisi potongan miliknya, mis. "AI" + "Kamu potongan ke-3"
- Peserta saling teriak potongan untuk merakit "SAFE9AI24"

Validasi saat publish (bukan saat live). CMS wajib memastikan gabungan seluruh fragments (f1..f4 = "SAFE"+"9"+"AI"+"24") sama persis dengan expected "SAFE9AI24". Kalau author salah ketik, publish gagal — bukan ketahuan di depan 18 peserta.

PR yang belum diputus (dari blueprint): aturan pengurutan potongan. Saat peserta reconnect, posisinya di daftar presence bisa berubah → potongannya bisa tertukar. Bekukan urutan saat fase 7a dimulai.

RTDB berubah:

```
/sessions/{id}/live/fragmentOrder = ["p_aud","p_ben","p_cah","p_dian"] // dibekukan saat 7a mulai
/sessions/{id}/live/codeinput/phase-7b = { attempts: 2, solved: true, solvedAt }
/sessions/{id}/live/phasePointer = { activePhaseId: "phase-summary" } // onSuccess.advance
```

Apa yang dibuktikan dokumen ini

Ini bukan sekadar contoh konten — ini uji kelayakan schema. Empat hal terkonfirmasi:

1. Semua tipe fase muat dalam satu struktur Phase

Delapan fase yang sangat berbeda (presentasi terpimpin, modul mandiri, kuis Kahoot, refleksi tenang, mini-game, kunci kolaboratif) semuanya jadi objek Phase yang valid. Tidak ada fase yang "tidak muat". Kalau nanti ada ide fase baru yang tidak bisa ditulis begini, itu sinyal schema perlu diperluas — dan lebih baik ketahuan lewat latihan seperti ini.

2. syncMode per fase benar-benar diperlukan

Fase 4 (lockstep) dan fase 5 (self_paced) sama-sama bertipe "quiz" tapi terasa seperti dua fitur berbeda. Kalau progres pemain dipaksa satu model global, salah satu dari keduanya mustahil dibuat. Inilah pembenaran konkret jawabanmu "per-phase configurable".

3. Pemisahan Firestore vs RTDB terlihat jelas

Seluruh JSON di dokumen ini adalah konten beku di publishedGames (Firestore). Yang berdenyut saat sesi hanyalah blok **RTDB berubah** di tiap fase — kecil, tercakup sempit, dan tidak pernah menyentuh konten. Itulah yang menjaga biaya rendah & sinkronisasi cepat.

4. Dua risiko nyata tetap kelihatan

Kebocoran kunci jawaban (fase 4) dan pengurutan potongan saat reconnect (fase 7) tidak "hilang" hanya karena ini contoh. Keduanya sengaja ditandai supaya kamu ingat memutuskannya sebelum ngoding.

Langkah lanjut yang masuk akal: pilih 3 fase paling representatif dari dokumen ini (saran: fase 2 microlearning, fase 4 quiz-live, fase 7 code) sebagai target vertical-slice pertama. Kalau ketiganya jalan end-to-end di 3 perangkat, sisa fase hanyalah menambah renderer — arsitektur intinya sudah terbukti.